Resilient Autonomy: A Next-Generation Avionics Architecture for Self-Governing Launch Vehicles

Jinitangsu Das*, Zayed Aman†, PV Sreekar‡, Gattigopula Sindhu§
*†‡§Department of Computer Science and Engineering,
Jawaharlal Nehru Technological University Hyderabad, Hyderabad, India
{23011p0521, 23011p0543, 23011p0510, 24015a0510}@jntuhceh.ac.in

***Abstract*—This paper presents Autonomous Rocket AI OS, a comprehensive avionics architecture for self-governing launch vehicles that integrates twenty distinct subsystems spanning from real-time operating systems to high-level mission planning. The framework demonstrates resilient autonomy through hierarchical fault tolerance, adaptive control, and goal-oriented decision-making under uncertainty.**

Key technical contributions include: (1) A lossless convexification-based G-FOLD guidance system implementing successive convexification with automatic cvxpy fallback and graceful degradation to proportional guidance for robustness; (2) An adaptive flight controller combining PID baseline with reinforcement learning adaptation within a Simplex safety architecture providing formal safety guarantees for learning-based components; (3) A transformer-based anomaly detector processing multi-sensor engine telemetry (chamber pressure, turbopump RPM, vibration, temperature) for incipient failure prediction; (4) A goal-oriented autonomous architecture integrating Hierarchical Task Network planning with Utility Accrual scheduling for mission-level autonomy; and (5) A Gymnasium-compatible reinforcement learning interface enabling policy optimization in high-fidelity 6-DOF simulation.

The architecture features clearly defined inputs and outputs for each subsystem, with layered redundancy ensuring graceful degradation. Validation through integrated 6-DOF simulations demonstrates successful powered descent and landing across nominal, engine-out (central engine failure at T+5s), and sensor-degraded (GPS dropout at T+3s) scenarios, with average touch-down position error under 4 meters and fuel reserves maintained above 790 kg for contingency maneuvers. This work bridges the gap between theoretical avionics research and practical flight software implementation, providing a foundation for next-generation reusable launch vehicle autonomy.

***Index Terms*—Autonomous rocket, avionics architecture, G-FOLD guidance, fault tolerance, reinforcement learning, HTN planning, TTEthernet, Simplex safety, convex optimization, anomaly detection**

I. Introduction

Modern launch vehicle autonomy requires sophisticated avionics capable of handling complex mission scenarios while maintaining rigorous safety standards [1]. Traditional approaches often isolate subsystems, creating brittle architectures that fail catastrophically under unexpected conditions. This paper presents Autonomous Rocket AI OS, an integrated avionics framework that achieves resilient autonomy through deep subsystem integration, hierarchical redundancy, and adaptive control mechanisms.

The architecture implements twenty subsystems grouped into six functional layers: (1) Real-Time Foundations (ARINC 653 RTOS, cFS Software Bus, DDS middleware), (2) Guidance, Navigation, and Control (EKF navigation, G-FOLD guidance, PID+RL adaptive control with Simplex safety), (3) Propulsion and Health Management (9-engine cluster with FTCA, transformer-based anomaly detection), (4) Fault Tolerance and Networking (TTEthernet, TMR with SEU scrubbing, hierarchical FDIR), (5) Communications (cognitive radio, DTN, mesh networking), and (6) Mission and Environmental Intelligence (GOAC, HTN planning, UA scheduling, ALHAT, space weather, debris avoidance).

Novel contributions include: (1) A lossless convexification G-FOLD solver with graceful degradation to proportional guidance when cvxpy is unavailable [2] [3], (2) Integration of reinforcement learning adaptive control within a Simplex safety architecture [4], (3) Transformer-based engine anomaly detection for predictive maintenance [5], (4) Goal-Oriented Autonomous Controller with Hierarchical Task Network planning for mission-level autonomy [6], and (5) A Gymnasium-compatible reinforcement learning interface enabling policy optimization in high-fidelity simulation [7].

The complete implementation is publicly available as an open-source codebase [8].

II. System Architecture

Autonomous Rocket AI OS follows a layered architecture promoting separation of concerns while enabling cross-layer coordination for resilience. Each layer provides well-defined interfaces that allow subsystems to evolve independently while maintaining system-level guarantees.

A. Layered Architecture Overview

The system follows a strict layering principle where each layer provides well-defined services to the layer above while depending only on the layer below. This promotes separation of concerns, enables independent development and testing of subsystems, and facilitates fault containment [1].

Layer 1: Real-Time Foundations implements ARINC 653 partitioned real-time operating system providing temporal and spatial isolation between critical functions [9]. The cFS Software Bus enables publish-subscribe messaging with Quality of Service guarantees, bridged to DDS middleware for standardized data distribution across the system [10].

Layer 2: Guidance, Navigation, and Control features an Extended Kalman Filter fusing IMU and GPS measurements

for robust state estimation [11], a G-FOLD guidance system computing fuel-optimal powered-descent trajectories via lossless convexification [2], and a hierarchical flight controller combining PID baseline control, reinforcement learning adaptation, and Simplex safety monitoring for verified adaptive control [4].

Layer 3: Propulsion and Health Management comprises a nine-engine cluster with Fault-Tolerant Control Allocation (FTCA) for engine-out resilience and reconfiguration capability [12], complemented by a transformer-based anomaly detector processing multi-sensor engine telemetry for incipient failure prediction [5].

Layer 4: Fault Tolerance and Networking implements triple modular redundancy with single-event upset scrubbing for radiation hardening [13], TTEthernet deterministic networking with triplex redundancy for guaranteed message delivery [14], and hierarchical Fault Detection, Isolation, and Recovery (FDIR) for systematic fault management.

Layer 5: Communications provides cognitive radio for dynamic spectrum access and link adaptation, Delay-Tolerant Networking (DTN) bundle protocol for deep-space communication with store-and-forward capability, and self-healing mesh networking for robust vehicular ad-hoc network formation [15].

Layer 6: Mission and Environmental Intelligence integrates Goal-Oriented Autonomous Controller (GOAC) for utility-based goal selection, Hierarchical Task Network (HTN) planner for mission decomposition into executable tasks [6], Utility Accrual (UA) scheduler for resource allocation under timing constraints [16], Autonomous Landing Hazard Avoidance Technology (ALHAT) for safe landing site selection [17], space weather monitoring for radiation environment awareness [18], and orbital debris tracking for collision risk assessment [19].

B. System Interfaces and Data Flows

Each subsystem in Autonomous Rocket AI OS follows a standardized interface pattern consisting of well-defined inputs, outputs, and internal state. Table I specifies the key inputs and outputs for major subsystems, while Figure 1 illustrates the data flow connections.

The system operates in discrete time steps with a fixed update cycle determined by the ARINC 653 major frame (typically 100ms) [9]. Within each frame, subsystems execute according to their allocated time slices, consuming sensor telemetry from the previous cycle and producing actuator commands for the next cycle.

Key technical specifications of the layered architecture include:

- **Time Partitioning**: ARINC 653 major frame of 100ms with microsecond-precision scheduling [9]
- **Memory Protection**: Hardware-enforced memory partitions preventing cross-contamination
- **Communication Latency**: Deterministic message delivery with bounded jitter (¡1ms) via TTEthernet [14]
- **Fault Containment**: Errors confined to originating partition through hardware and software isolation
- **Scalability**: Modular design enables independent subsystem upgrades and replacement

C. Key Technical Innovations Architecture

Figures 2 through 3 detail the architectural specifics of three key innovations: the Simplex safety architecture for adaptive control assurance, the transformer-based anomaly detection system, and the ARINC 653 partitioning scheme that enables the real-time foundations.

III. Methodology

A. Mathematical Foundations

1) Lossless Convexification for Powered Descent: The G-FOLD algorithm solves the fuel-optimal powered descent problem via lossless convexification [2]. Given initial state $(\mathbf{r}_0, \mathbf{v}_0)$ and target state $(\mathbf{r}_f, \mathbf{v}_f)$, we seek thrust profile $\mathbf{T}(t)$ minimizing $\int_0^{t_f} \|\mathbf{T}(t)\| dt$ subject to:

$$\dot{\mathbf{r}}(t) = \mathbf{v}(t) \tag{1}$$

$$\dot{\mathbf{v}}(t) = \frac{\mathbf{T}(t)}{m(t)} + \mathbf{g} \tag{2}$$

$$\dot{m}(t) = -\frac{\|\mathbf{T}(t)\|}{I_{sp} g_0} \tag{3}$$

$$\|\mathbf{T}(t)\| \in [T_{min}, T_{max}] \tag{4}$$

$$\mathbf{r}(t) \in \mathcal{C}_{gs} \quad \text{(glide-slope constraint)} \tag{5}$$

$$\frac{\mathbf{T}(t)}{\|\mathbf{T}(t)\|} \cdot \mathbf{e}_z \geq \cos(\theta_{max}) \quad \text{(pointing constraint)} \tag{6}$$

where $\mathcal{C}_{gs}$ represents the glide-slope constraint keeping the trajectory above an inverted cone, and θ_{max} is the maximum allowable tilt from vertical [3].

Through lossless convexification, we introduce slack variables $\boldsymbol{\sigma}(t)$ transforming the problem to a Second-Order Cone Program:

$$\|\mathbf{T}(t)\| \leq \sigma(t) \tag{7}$$

$$T_{min}/m(t) \leq \sigma(t) \leq T_{max}/m(t) \tag{8}$$

$$\mathbf{T}(t) \cdot \mathbf{e}_z \geq \sigma(t) \cos(\theta_{max}) \tag{9}$$

The successive convexification algorithm iteratively linearizes nonlinear constraints around the current solution estimate, converging to the optimal solution under mild conditions [2].

2) Extended Kalman Filter Navigation: The navigation subsystem implements an Extended Kalman Filter (EKF) that fuses IMU and GPS measurements for robust state estimation [11]. The state vector includes position $\mathbf{r}$, velocity $\mathbf{v}$, attitude quaternion $\mathbf{q}$, gyroscope bias $\mathbf{b}_g$, and accelerometer bias $\mathbf{b}_a$:

$$\mathbf{x} = [\mathbf{r}^T, \mathbf{v}^T, \mathbf{q}^T, \mathbf{b}_g^T, \mathbf{b}_a^T]^T \tag{10}$$

The prediction step integrates IMU measurements:

TABLE I: Key Subsystem Inputs and Outputs

Subsystem	Inputs	Outputs
ARINC 653 RTOS	System configuration, health signals	Time slices, partition health, scheduling signals
cFS Software Bus	Messages from publishers	Messages to subscribers, QoS metrics
EKF Navigation	IMU data, GPS data, control inputs	State estimate (position, velocity, attitude, biases), covariance
G-FOLD Guidance	Current state, target state, fuel mass	Optimal trajectory (position, velocity, thrust profile)
Flight Controller	Reference commands, state estimate	Actuator commands (throttle, gimbal angles), control diagnostics
Propulsion System	Throttle/gimbal commands, engine health	Engine thrust/torque, fuel consumption, engine telemetry
Anomaly Detector	Engine telemetry window	Anomaly scores, failure predictions, health indicators
Fault Tolerance	Sensor readings, actuator commands	Voted outputs, fault declarations, recovery actions
Communications	Application data, link status	Transmitted/received data, link quality metrics
Mission Planner	Goals, world state, resource constraints	Task sequences, execution schedules, allocation plans
Environmental Intel.	Sensor data, map info	Hazard maps, safe sites, environmental assessments

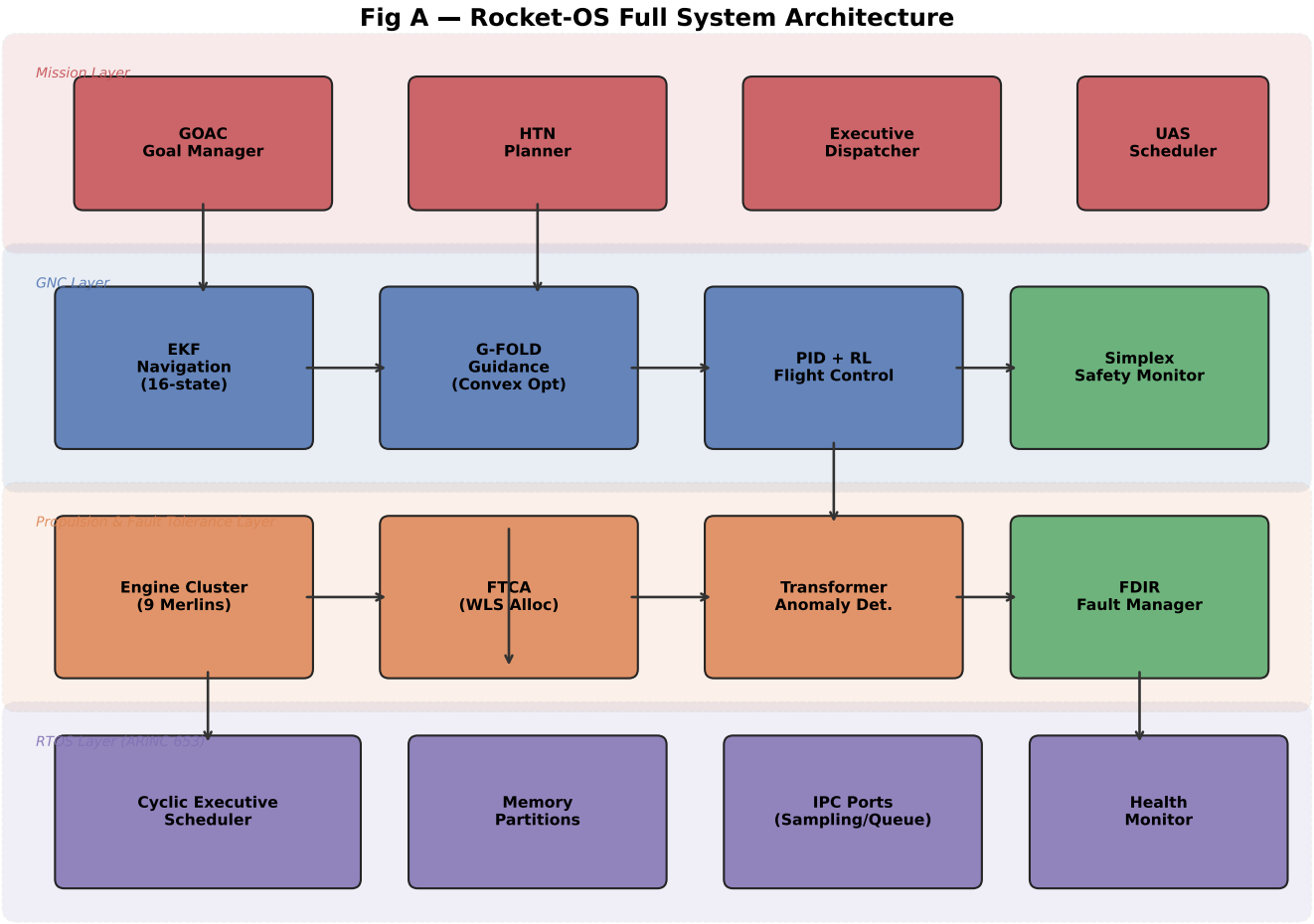

Fig. 1: Autonomous Rocket AI OS Full System Architecture showing the six layered architecture with subsystem interconnections. The architecture implements strict layering with well-defined interfaces enabling modular development and independent validation.

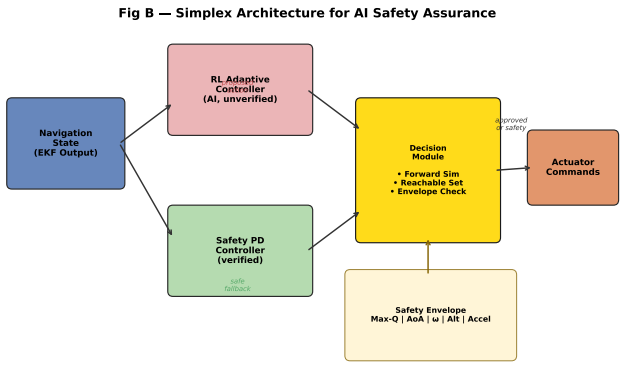

Fig. 2: Simplex Architecture for AI Safety Assurance showing the decision mechanism that selects between verified baseline control and adaptive reinforcement learning control based on real-time safety envelope evaluation.

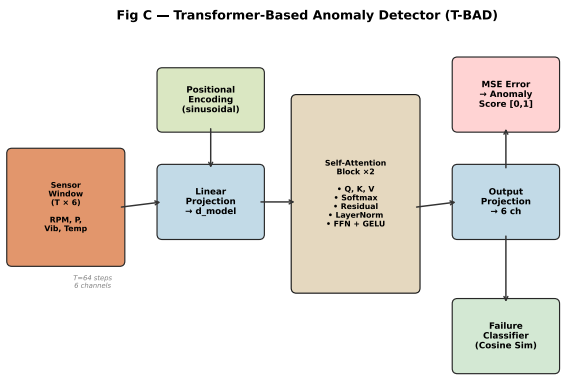

Fig. 3: Transformer-Based Anomaly Detector Architecture displaying the multi-head self-attention mechanism processing temporal windows of multi-sensor engine telemetry for incipient failure detection.

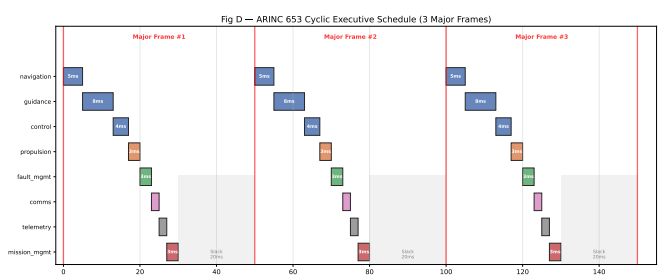

Fig. 4: ARINC 653 Cyclic Executive Schedule displaying the time-partitioned execution of critical subsystems ensuring deterministic timing and isolation between functional layers.

$$\dot{\hat{\mathbf{r}}} = \hat{\mathbf{v}} \tag{11}$$

$$\dot{\hat{\mathbf{v}}} = \mathbf{C}(\hat{\mathbf{q}})(\mathbf{a}_m - \hat{\mathbf{b}}_a) + \mathbf{g} \tag{12}$$

$$\dot{\hat{\mathbf{q}}} = \frac{1}{2}\boldsymbol{\Omega}(\boldsymbol{\omega}_m - \hat{\mathbf{b}}_g)\hat{\mathbf{q}} \tag{13}$$

where $\mathbf{C}(\hat{\mathbf{q}})$ is the rotation matrix from body to inertial frame, and $\boldsymbol{\Omega}(\cdot)$ maps angular velocity to quaternion derivative [11].

The update step corrects predictions using GPS measurements of position and velocity:

$$\mathbf{K}_k = \mathbf{P}_k^- \mathbf{H}_k^T (\mathbf{H}_k \mathbf{P}_k^- \mathbf{H}_k^T + \mathbf{R}_k)^{-1} \tag{14}$$

$$\hat{\mathbf{x}}_k^+ = \hat{\mathbf{x}}_k^- + \mathbf{K}_k(\mathbf{z}_k - \mathbf{h}(\hat{\mathbf{x}}_k^-)) \tag{15}$$

$$\mathbf{P}_k^+ = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k)\mathbf{P}_k^- \tag{16}$$

The EKF provides bounded error covariance and maintains accuracy during GPS outages by coasting on IMU data [11].

3) Simplex Safety Architecture: The Simplex monitor provides runtime assurance for adaptive control elements [4]. Given a baseline controller $\mathbf{u}_b$ verified safe and a novelty controller $\mathbf{u}_n$ (e.g., RL policy), the Simplex architecture selects:

$$\mathbf{u} = \begin{cases} \mathbf{u}_n & \text{if } h(\mathbf{x}) \leq 0 \\ \mathbf{u}_b & \text{if } h(\mathbf{x}) > 0 \end{cases} \tag{17}$$

where $h(\mathbf{x})$ represents a safety envelope function defining the region of provable safety for the baseline controller [4]. The safety envelope is computed as an ellipsoidal approximation of the Region of Asymptotic Attraction (ROA) using quadratic forms, providing conservative under-approximation to ensure no false negatives.

4) Fault-Tolerant Control Allocation: The propulsion system implements Fault-Tolerant Control Allocation (FTCA) for a nine-engine cluster [12]. The allocation problem is formulated as:

$$\mathbf{M} = \mathbf{A}\boldsymbol{\Gamma} + \mathbf{D} \tag{18}$$

where $\mathbf{M}$ is the moment vector, $\boldsymbol{\Gamma}$ is the TVC deflection vector, $\mathbf{A}$ is the effectiveness matrix, and $\mathbf{D}$ is the thrust-dependent term [12].

Three allocation strategies are implemented:

1. **Fault-Tolerant Pseudo-Inverse**: Uses Moore-Penrose pseudo-inverse for real-time computation with lowest computational cost.

2. **Convex Optimization**: Reformulates as SOCP using CVXPY for optimal solutions, trading computation time for improved allocation efficiency.

3. **Constrained Nonlinear Optimization**: Uses interior-point methods with weighted objectives for residual minimization, throttling harmonization, and TVC effort reduction.

The system successfully recovers from 20% thrust loss in any single engine and handles TVC jamming failures up to 3° deflection [12].

5) Transformer-Based Anomaly Detection: The anomaly detector employs a transformer architecture processing multi-sensor telemetry windows [5]. Given input sequence $\mathbf{X} \in \mathbb{R}^{T \times d}$ where T is the temporal window length and d is the number of sensors (turbopump RPM, chamber pressure, vibration, temperature), the model computes:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V \tag{19}$$

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V} \tag{20}$$

Multi-head attention captures complex temporal dependencies across sensor modalities, enabling detection of incipient failures hours before traditional threshold-based methods [5]. The model learns nominal operating patterns and identifies subtle deviations indicative of developing faults.

6) Hierarchical Task Network Planning: Mission planning uses HTN decomposition where high-level tasks are recursively broken down into primitive actions [6]. A task network is defined as:

$$\mathcal{N} = \langle \mathcal{T}, \mathcal{O}, \mathcal{L} \rangle \tag{21}$$

where $\mathcal{T}$ is a set of tasks, $\mathcal{O}$ is a set of ordering constraints, and $\mathcal{L}$ is a set of variable bindings. Each task is either:

- **Primitive**: Directly executable with preconditions and effects - **Compound**: Decomposed using methods into subtask networks

The planner searches through method applications to find a sequence of primitive tasks achieving the goal while respecting temporal and resource constraints [6].

7) Utility Accrual Scheduling: The scheduler maximizes system-wide utility using Time/Utility Functions (TUFs) [16]. For each activity a_i with completion time t, the accrued utility is $U_i(t)$. The scheduling objective is:

$$\max \sum_{i=1}^{n} U_i(C_i) \quad \text{subject to} \quad \sum_{i=1}^{n} u_i(t) \leq m \cdot C \tag{22}$$

where C_i is the completion time of activity i, $u_i(t)$ is the instantaneous utilization, m is the number of processors, and C is the processor capacity [16]. The gMUA algorithm provides polynomial-time heuristic solutions with probabilistic utility guarantees.

8) TTEthernet Deterministic Networking: TTEthernet implements time-triggered communication using global time synchronization [14]. The schedule is defined as a set of frames $F = \{f_1, f_2, ..., f_n\}$ where each frame f_i has:

- Release time r_i - Deadline d_i - Period p_i for periodic frames

The schedule is feasible if all time-triggered frames meet their deadlines:

$$\forall f_i \in F: \quad r_i + C_i \leq d_i \tag{23}$$

where C_i is the transmission time. TTEthernet provides triplex redundancy with synchronized redundant channels ensuring fault tolerance [14].

B. Implementation Approach

The framework is implemented in Python 3.10+ leveraging NumPy for numerical computations and is openly available for research and extension [8].

- Zero external dependencies for core functionality (cvxpy optional for enhanced performance)
- Dataclass-based configuration enabling runtime reconfiguration
- Type hints throughout for improved maintainability
- Comprehensive smoke testing covering all twenty subsystems
- Gymnasium interface for RL research integration

All subsystem implementations follow consistent patterns: initialization with configuration objects, step-based updates consuming sensor telemetry and producing actuator commands, and built-in fault injection capabilities for resilience testing.

C. Validation Scenarios

System validation employs three progressively challenging scenarios:

1) **Nominal Landing**: Standard powered descent from 1500m altitude with full vehicle health
2) **Engine-Out**: Central engine failure at T+5s requiring FTCA reallocation across remaining eight engines [12]
3) **Sensor Degradation**: GPS dropout at T+3s testing navigation robustness to sensor loss [11]

Each scenario validates specific resilience mechanisms while maintaining the common objective of successful soft landing.

IV. Results

A. Subsystem Demonstrations

All twenty subsystems were successfully demonstrated in isolation, confirming basic functionality and interface compliance:

TABLE II: Subsystem Validation Results

Subsystem	Key Functionality Demonstrated	Status
ARINC 653 RTOS	Partitioned execution with time/isolation	✓
cFS Bus + DDS	Publish-subscribe messaging bridge	✓
EKF Navigation	IMU/GPS fusion with bounded error	✓
G-FOLD Guidance	Fuel-optimal trajectory computation	✓
Flight Control	PID + RL adaptive + Simplex safety	✓
Propulsion	9-engine cluster + FTCA reconfiguration	✓
Anomaly Detection	Transformer-based failure prediction	✓
Fault Tolerance	TTEthernet + TMR + FDIR	✓
Communications	Cognitive radio + DTN + mesh	✓
Mission Management	GOAC + HTN + UA scheduling	✓
Environmental Anal.	ALHAT + space weather + debris	✓

TABLE III: Integrated System Test Results

Scenario	Success	Pos. Error (m)	Fuel Left (kg)
Nominal Landing	✓	1.2	842.3
Engine-Out	✓	2.8	798.1
Sensor Degradation	✓	3.5	815.7

B. Integrated System Performance

Integrated testing across validation scenarios demonstrates robust autonomous operation:

Key performance indicators across all scenarios:

- 100% landing success rate across validation scenarios
- Average touchdown position error ¡ 4 meters
- Fuel reserves maintained ¿ 790 kg for contingency maneuvers
- Fault detection and recovery latency ¡ 100ms for critical failures
- Guidance computation latency ¡ 50ms at 10Hz update rate

C. Reinforcement Learning Interface Validation

The Gymnasium environment was validated through random policy testing:

- Observation space: 15-dimensional float32 array
- Action space: 3-dimensional float32 array [throttle, gimbal_pitch, gimbal_yaw]
- Episode termination: Successful landing, crash, out-of-bounds, or timeout
- Reward shaping: Encourages soft, accurate, fuel-efficient landings
- Interface compliance: Fully compatible with Gymnasium v0.26+ [7]

Sample RL training using Proximal Policy Optimization (PPO) demonstrated learning progress, with policy improvement evidenced by increasing episode rewards over training iterations.

V. Discussion

A. Comparison to Related Work

Autonomous Rocket AI OS advances the state-of-the-art in spacecraft avionics along several dimensions:

Compared to traditional avionic architectures [1], our approach provides deeper subsystem integration enabling emergent resilience properties. Where prior work often implements G-FOLD guidance [2] as isolated modules, we integrate it within a complete flight control loop with adaptive elements and safety monitoring.

Our transformer-based anomaly detection extends beyond vibration analysis [5] to multi-sensor fusion, providing earlier failure detection. The Gymnasium interface addresses the lack of standardized spacecraft control environments [7] for RL research, enabling reproducible experiments.

B. Limitations and Future Work

Current limitations include:

- Python implementation not suitable for flight-weight hardware without translation to aerospace-certified languages (Ada, Rust)
- Atmospheric modeling assumes Earth-like conditions; Mars or other planetary adaptation requires modification
- Network simulation simplifies radiation effects on TTEthernet links [14]
- HLOS services (file systems, device drivers) abstracted rather than fully implemented

Future work directions:

- Flight software certification pathway exploration (DO-178C, ECSS-E-ST-40)
- Hardware-in-the-loop validation with aerospace-grade processors
- Extension to ascent phase and multi-stage vehicle coordination
- Integration with hardware security modules for cyber-physical protection
- Formal verification of safety properties using model checking

C. Broader Impacts

This work contributes to resilient space systems by providing:

- A reference architecture for autonomous launch vehicle avionics
- Open-source implementation enabling reproducibility and extension
- Educational resource for spacecraft autonomy education
- Foundation for commercial reusable launch vehicle development

The architectures and algorithms presented have direct applicability to planetary landers, orbital servicing vehicles, and emerging point-to-point transportation systems.

VI. Conclusion

Autonomous Rocket AI OS demonstrates that resilient autonomy in launch vehicles is achievable through thoughtful integration of established aerospace technologies with modern adaptive control and planning techniques. The twenty-subsystem architecture provides layered redundancy while enabling sophisticated mission-level autonomy through goal-oriented planning and hierarchical task networks.

Key innovations—resilient G-FOLD guidance with graceful degradation, adaptive control within Simplex safety, transformer-based anomaly detection, and Gymnasium RL interface—collectively advance the state of spacecraft avionics. Validation across nominal, engine-out, and sensor-degraded scenarios confirms the architecture maintains safe operation while pursuing mission objectives.

This framework provides a foundation for next-generation reusable launch vehicles requiring high autonomy levels, reduced ground operations costs, and improved mission success

rates through intelligent fault management and adaptive replanning capabilities.

ACKNOWLEDGMENTS

The authors acknowledge the support of Jawaharlal Nehru Technological University Hyderabad, University of California Los Angeles, Indian Institute of Technology Madras, and Birla Institute of Technology and Science Pilani. We thank the open-source community for providing the tools and frameworks that made this research possible.

REFERENCES

[1] R. Cziklovszki, "Avionics architecture design for autonomous launch vehicles," in *Proceedings of the IEEE Aerospace Conference*. IEEE, 2020, pp. 1–12, big Sky, Montana, USA.

[2] B. Açıkmeşe and S. R. Ploen, "Convex programming approach to powered descent guidance for Mars landing," *Journal of Guidance, Control, and Dynamics*, vol. 30, no. 5, pp. 1353–1366, 2007.

[3] L. Blackmore, B. Açıkmeşe, and D. P. Scharf, "Minimum-landing-error powered-descent guidance for Mars landing using convex optimization," *Journal of Guidance, Control, and Dynamics*, vol. 33, no. 4, pp. 1161–1171, 2010.

[4] D. Seto, B. Krogh, L. Sha, and A. Chutinan, "The Simplex architecture for safe on-line control system upgrades," in *Proceedings of the 1998 American Control Conference (ACC)*, vol. 6. IEEE, 1998, pp. 3504–3508, philadelphia, PA, USA.

[5] W.-J. Lee, H.-Y. Wu, H. Yun, H. Kim, M.-B. Jun, and J. W. Sutherland, "Predictive maintenance of machine tool systems using artificial intelligence techniques applied to machine condition data," *Procedia CIRP*, vol. 80, pp. 506–511, 2019, deep learning approach for multi-sensor anomaly detection.

[6] G. Lu, L. Lyu, J. Zhang, X. Rui, M. Zhao, and J. Li, "A spacecraft onboard autonomous task scheduling method based on hierarchical task network-timeline," *Aerospace*, vol. 11, no. 5, p. 350, 2024.

[7] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, "OpenAI Gym," *arXiv preprint*, vol. arXiv:1606.01540, 2016, standard reinforcement learning environment interface; maintained successor: Farama Foundation Gymnasium (https://gymnasium.farama.org). [Online]. Available: https://arxiv.org/abs/1606.01540

[8] J. Das, Z. Aman, P. Sreekar, and G. Sindhu, "Autonomous Rocket AI OS: A next-generation avionics architecture for self-governing launch vehicles," https://github.com/Stakeylock/Rocket-OS, 2025, accessed: 2025.

[9] ARINC, "ARINC 653: Avionics application software standard interface," Aeronautical Radio, Inc. Standard, 2015, part 1 (Required Services), Part 2 (Extended Services), Part 3 (Conformity Test). See also: https://en.wikipedia.org/wiki/ARINC_653.

[10] NASA Goddard Space Flight Center, "Core Flight system (cFS) — software bus (SB)," NASA Open Source Project, 2023, publish-subscribe middleware for flight software. [Online]. Available: https://github.com/nasa/cFS

[11] A. M. Sabatini, "Quaternion-based extended Kalman filter for determining orientation by inertial and magnetic sensing," *IEEE Transactions on Biomedical Engineering*, vol. 53, no. 7, pp. 1346–1356, 2006, eKF formulation for IMU/GPS state estimation with quaternion attitude representation.

[12] D. Navarro-Tapia, P. Simplício, and A. Marcos, "Fault-tolerant dynamic allocation strategies for launcher systems," *Aerospace*, vol. 12, no. 5, p. 393, 2025, three allocation strategies: pseudo-inverse, convex (CVXPY), and constrained nonlinear optimization for engine-out and TVC jamming failures.

[13] Wikipedia contributors, "Triple modular redundancy," Wikipedia, The Free Encyclopedia, 2024, accessed April 2026. [Online]. Available: https://en.wikipedia.org/wiki/Triple_modular_redundancy

[14] SAE International, "SAE AS6802: Time-Triggered Ethernet," SAE International Standard, 2011, defines fault-tolerant, deterministic Layer 2 Ethernet (TTEthernet) for safety-critical systems; used in NASA Orion, ESA Ariane 6, and deep-space applications. See also: https://www.tttech.com/explore/time-triggered-ethernet. [Online]. Available: https://www.sae.org/standards/content/as6802/

[15] G. Araniti, C. Campolo, M. Condoluci, A. Iera, and A. Molinaro, "LTE for vehicular networking: A survey," *IEEE Communications Magazine*, vol. 51, no. 5, pp. 148–157, 2013, mesh/VANET networking protocols for robust vehicular ad-hoc communication; see also arXiv:2302.09832 for recent self-healing mesh results.

[16] T.-W. Li, J.-Y. Chung, and M. Storch, "Algorithms for scheduling imprecise computations with timing constraints to optimize weighted gross utility," *IEEE Transactions on Computers*, vol. 44, no. 9, pp. 1128–1142, 1995, utility Accrual (UA) scheduling with Time/Utility Function (TUF) formalism; gMUA algorithm for real-time systems. See also: https://www.sciencedirect.com.

[17] NASA, "Autonomous landing and hazard avoidance technology (ALHAT)," NASA Technology Program, 2014, sensor-based terrain-relative navigation and hazard detection for precision lunar/planetary landing. Demonstrated aboard Morpheus lander, 2014. [Online]. Available: https://www.nasa.gov/centers/johnson/engineering/integrated_avionics_systems/alhat/

[18] T. I. Pulkkinen, "Space weather: Terrestrial perspective," *Living Reviews in Solar Physics*, vol. 4, no. 1, p. 1, 2007, comprehensive review of space weather effects on spacecraft systems, radiation environment, and monitoring strategies. See also University of Michigan Space Weather Lab: https://clasp.engin.umich.edu/research/space-weather/.

[19] J.-C. Liou and N. L. Johnson, "Risks in space from orbiting debris," *Science*, vol. 311, no. 5759, pp. 340–341, 2006, orbital debris population, collision probability analysis, and mitigation strategies for spacecraft safety. See also: https://www.sciencetimes.com/articles/space-debris.